

02 — Character Data Types in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Overview of Character Types](#)
 3. [char\(n\) — Fixed-Length](#)
 4. [varchar\(n\) — Variable-Length with Limit](#)
 5. [text — Unlimited Variable-Length](#)
 6. [Storage Internals \(TOAST\)](#)
 7. [Character Set & Encoding](#)
 8. [Collation](#)
 9. [String Functions Reference](#)
 10. [SQL Examples](#)
 11. [Performance Comparison](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Exercises with Solutions](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain the technical difference between `char(n)`, `varchar(n)`, and `text`
 - Understand how TOAST compresses and stores large strings
 - Configure and use collation for correct locale-aware sorting and comparison
 - Choose the right character type for every use-case
 - Avoid padding surprises with `char(n)`
 - Use PostgreSQL's rich set of string functions effectively
-

Overview of Character Types

Character Type Family

```
|— character(n) / char(n)      — fixed-length, blank-padded
|— character varying(n) / varchar(n) — variable-length, max n chars
|— text                        — variable-length, no stated limit
```

Critical insight: In PostgreSQL, `varchar(n)` and `text` use the **same internal storage mechanism**. The only difference is that `varchar(n)` enforces a length check at insert/update time. This is fundamentally different from many other databases where these types have different storage engines.

char(n) — Fixed-Length

Definition

`CHARACTER(n)` or `CHAR(n)` stores strings of exactly `n` characters. If the stored string is shorter than `n`, it is **right-padded with spaces**.

Storing 'AB' in `char(5)`:

A	B			
---	---	--	--	--

 ← padded with 3 spaces

Internal representation: 'AB ' (5 chars)

Comparison behavior

Trailing spaces are **ignored** in comparisons for `char(n)`:

```
SELECT 'AB'::CHAR(5) = 'AB'::CHAR(2); -- TRUE (spaces stripped for comparison)
SELECT 'AB ' = 'AB';                  -- TRUE when using char type
```

When to use `char(n)`

- Fixed-length codes where every value is exactly `n` characters: ISO country codes (`CHAR(2)`), currency codes (`CHAR(3)`), US ZIP codes (`CHAR(5)`)
- Legacy compatibility only

Storage cost

A `CHAR(n)` column always stores `n` characters worth of bytes (plus 1–4 bytes overhead depending on length) — even if the actual value is shorter.

`varchar(n)` — Variable-Length with Limit

Definition

`CHARACTER VARYING(n)` or `VARCHAR(n)` stores strings up to `n` characters. Unlike `char(n)`, it does **not** pad shorter strings.

Storing 'AB' in `varchar(100)`:

'AB'

 ← stored as-is, 2 characters

Only 2 characters stored (+ overhead), not 100.

Length enforcement

```
INSERT INTO t (col) VALUES ('ABCDEFGH');
-- If col is varchar(5): ERROR: value too long for type character varying(5)
```

`varchar` without limit

`VARCHAR` with no parentheses is equivalent to `TEXT` — no length restriction.

text — Unlimited Variable-Length

Definition

TEXT stores any-length string without an explicit maximum. It is the **native PostgreSQL string type** and is often the best default choice.

text vs. varchar in PostgreSQL

Feature	text	varchar(n)
Storage mechanism	identical	identical
Performance	identical	identical
Max length	unlimited	n chars
Length check	none	enforced
SQL standard	no	yes

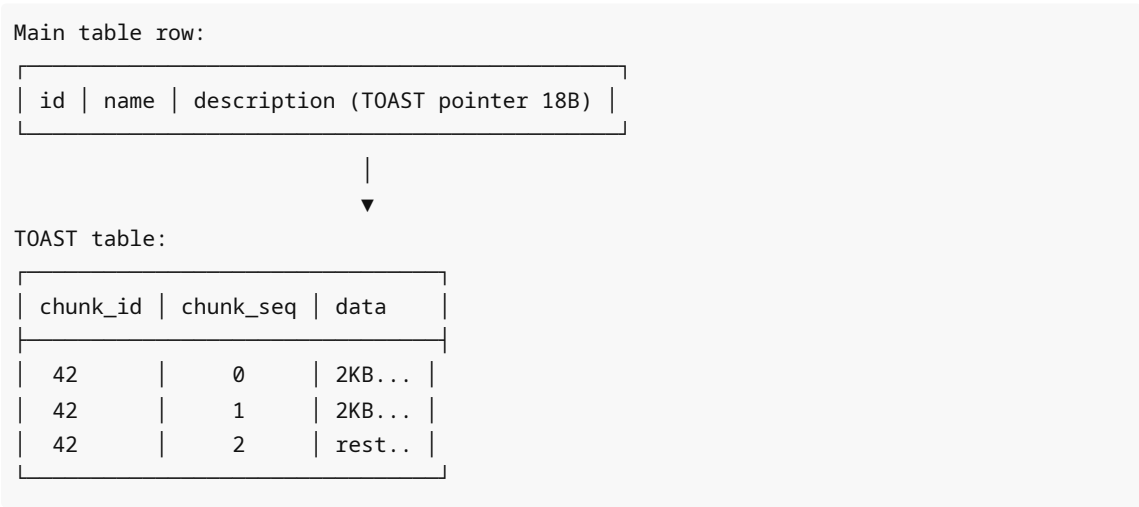
PostgreSQL documentation says: "There is no performance difference among these three types, apart from increased storage space when using the blank-padded type, and a few extra CPU cycles to check the length when storing into a length-constrained column."

Storage Internals (TOAST)

TOAST = **The Oversized-Attribute Storage Technique**

PostgreSQL's page size is 8 KB. A single row must fit on one page (with some exceptions). For large text values, TOAST transparently:

- 1. **Compresses** the value using LZ compression
- 2. **Chunks** the compressed value into 2 KB toast chunks
- 3. **Stores** chunks in a separate TOAST table
- 4. Stores a **pointer** (18 bytes) in the main table row



TOAST threshold: Values over ~2 KB trigger TOAST processing. **Impact:** Accessing a TOASTed column requires a separate I/O to the TOAST table — do not `SELECT *` when you only need small columns.

Character Set & Encoding

PostgreSQL databases have a single encoding set at creation time:

```
-- View current database encoding
SELECT pg_encoding_to_char(encoding) FROM pg_database WHERE datname =
current_database();

-- Create database with specific encoding
CREATE DATABASE myapp
  ENCODING = 'UTF8'
  LC_COLLATE = 'en_US.UTF-8'
  LC_CTYPE = 'en_US.UTF-8';
```

Common encodings:

Encoding	Description	Bytes per char
UTF8	Unicode (recommended)	1–4
LATIN1	ISO 8859-1 (Western European)	1
SQL_ASCII	No encoding checks performed	1
WIN1252	Windows CP1252	1

Best practice: Always use `UTF8`. It handles all languages, emoji, and special characters.

Collation

Collation determines the **sort order** and **comparison rules** for strings.

```
'a' < 'B' ? Depends on collation!
C collation:      'a' > 'B'  (ASCII order: lowercase > uppercase)
en_US.UTF-8:      'a' < 'B'  (locale-aware: a comes before B)
```

Column-level collation

```
CREATE TABLE products (
  product_name TEXT COLLATE "en_US.UTF-8",
  german_name  TEXT COLLATE "de_DE.UTF-8"
);

-- Query with explicit collation override
SELECT product_name
FROM products
ORDER BY product_name COLLATE "de_DE.UTF-8";
```

ICU collations (PostgreSQL 10+)

```
-- More portable, more features
CREATE TABLE names (
    full_name TEXT COLLATE "und-x-icu" -- Unicode default ordering
);
```

Case-insensitive matching

```
-- Using ILIKE (PostgreSQL extension)
SELECT * FROM products WHERE product_name ILIKE '%coffee%';

-- Using LOWER()
SELECT * FROM products WHERE LOWER(product_name) = LOWER('Coffee');

-- Using citext extension (case-insensitive text type)
CREATE EXTENSION IF NOT EXISTS citext;
CREATE TABLE users (
    email CITEXT NOT NULL UNIQUE -- uniqueness is case-insensitive
);
INSERT INTO users VALUES ('User@Example.com');
INSERT INTO users VALUES ('user@example.com'); -- ERROR: duplicate key
```

String Functions Reference

```
-- Length
SELECT length('Hello World');           -- 11 (chars)
SELECT octet_length('Hello');           -- 5 (bytes, differs for multibyte)
SELECT bit_length('Hello');             -- 40 (bits)
SELECT char_length('Héllö');            -- 5 (unicode chars)

-- Case conversion
SELECT upper('hello');                  -- 'HELLO'
SELECT lower('WORLD');                  -- 'world'
SELECT initcap('hello world');          -- 'Hello World'

-- Trimming
SELECT trim(' hello ');                 -- 'hello'
SELECT ltrim(' hello ');                -- 'hello'
SELECT rtrim(' hello ');                -- ' hello'
SELECT trim(BOTH 'x' FROM 'xxxhelloxxx'); -- 'hello'

-- Substring
SELECT substring('Hello World', 1, 5);  -- 'Hello'
SELECT left('Hello World', 5);          -- 'Hello'
SELECT right('Hello World', 5);         -- 'World'

-- Search & Replace
SELECT position('World' IN 'Hello World'); -- 7
SELECT strpos('Hello World', 'World');    -- 7
```

```

SELECT replace('Hello World', 'World', 'PostgreSQL'); -- 'Hello PostgreSQL'

-- Concatenation
SELECT 'Hello' || ' ' || 'World';           -- 'Hello World'
SELECT concat('Hello', ' ', 'World');       -- 'Hello World'
SELECT concat_ws(' ', 'Alice', 'Bob', 'Carol'); -- 'Alice, Bob, Carol'

-- Padding
SELECT lpad('42', 5, '0');                   -- '00042'
SELECT rpad('Hello', 10, '.');               -- 'Hello.....'

-- Splitting
SELECT split_part('a,b,c', ',', 2);          -- 'b'
SELECT string_to_array('a,b,c', ',');        -- {a,b,c}
SELECT array_to_string(ARRAY['a','b','c'], ','); -- 'a,b,c'

-- Regex
SELECT regexp_match('2024-01-15', '(\d{4})-(\d{2})-(\d{2})'); -- {2024,01,15}
SELECT regexp_replace('Hello World', 'World', 'PostgreSQL'); -- 'Hello PostgreSQL'
SELECT 'hello@example.com' ~ '^([a-z]+@[a-z]+\.[a-z]+)$'; -- TRUE

-- Format
SELECT format('Hello, %s! You are %s years old.', 'Alice', 30);

```

SQL Examples

Users table with various string columns

```

CREATE TABLE users (
  user_id          BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username         VARCHAR(50)     NOT NULL UNIQUE
                                CHECK (username ~ '^([a-zA-Z0-9_]){3,50}$'),
  email           VARCHAR(255)     NOT NULL UNIQUE,
  -- Use CHAR(2) for truly fixed-length codes
  country_code    CHAR(2)         NOT NULL DEFAULT 'US'
                                CHECK (country_code ~ '^([A-Z]){2}$'),
  -- Full name: text is fine, no arbitrary length limit needed
  full_name       TEXT            NOT NULL CHECK (length(full_name) >= 2),
  bio            TEXT,             -- nullable, any length
  password_hash   CHAR(60),        -- bcrypt hash is always exactly 60 chars
  locale         VARCHAR(10)      NOT NULL DEFAULT 'en-US',
  created_at     TIMESTAMPTZ      NOT NULL DEFAULT now()
);

-- Slugify a title (basic version)
CREATE OR REPLACE FUNCTION slugify(title TEXT)
RETURNS TEXT LANGUAGE SQL IMMUTABLE AS $$
  SELECT regexp_replace(
    lower(trim(title)),
    '^[a-z0-9]+', '-', 'g'

```

```

    );
$$;

-- Full-text search friendly column setup
CREATE TABLE articles (
  article_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title      TEXT      NOT NULL CHECK (length(title) BETWEEN 5 AND 500),
  slug       TEXT      NOT NULL UNIQUE GENERATED ALWAYS AS (slugify(title)) STORED,
  body       TEXT      NOT NULL,
  search_vec TSVECTOR GENERATED ALWAYS AS (
    to_tsvector('english', title || ' ' || body)
  ) STORED,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_articles_search ON articles USING GIN (search_vec);

```

Performance Comparison

Operation	char(n)	varchar(n)	text
Storage (short value)	n bytes	actual+1	actual+1
Storage (TOAST)	TOAST	TOAST	TOAST
Index lookup	same	same	same
Sort (short)	same	same	same
Sort (long, TOAST)	TOAST I/O	TOAST I/O	TOAST I/O
Length check on insert	pad	check n	none
Comparison (equality)	fast	fast	fast
Pattern match (LIKE)	same	same	same
Pattern match (index)	pg_trgm	pg_trgm	pg_trgm

Key takeaway: There is essentially no performance difference between `varchar(n)` and `text` in PostgreSQL. Choose based on business rules, not performance.

Common Mistakes

1. Using `char(n)` for variable-length data

```

-- BAD: 'Alice' stored as 'Alice      ' (padded to 20)
name CHAR(20)

-- GOOD: 'Alice' stored as 'Alice'
name TEXT

```

2. Comparing `char(n)` with `text` and getting surprises

```

SELECT 'AB'::CHAR(5) = 'AB'::TEXT; -- TRUE (spaces trimmed in comparison)
-- But:

```

```
SELECT 'AB'::CHAR(5) = 'AB   '::TEXT; -- TRUE! (trailing spaces ignored)
```

3. **Setting arbitrary varchar limits "just in case"** — `VARCHAR(255)` is often cargo-culted from MySQL. In PostgreSQL, use `TEXT` and enforce limits with CHECK constraints if needed.

4. **Not using `COLLATE` for locale-aware sorting**

```
-- German umlauts sort incorrectly without proper collation
SELECT name FROM people ORDER BY name;                -- wrong for
German
SELECT name FROM people ORDER BY name COLLATE "de_DE.UTF-8"; -- correct
```

5. **Forgetting TOAST overhead when selecting all columns**

```
-- BAD: pulls TOAST data for body even when you only need title
SELECT * FROM articles WHERE ...;

-- GOOD: only fetch what you need
SELECT article_id, title, created_at FROM articles WHERE ...;
```

Best Practices

1. **Default to `TEXT`** for string columns unless you have a specific reason for a limit.
 2. **Use `VARCHAR(n)` when** you want the database to enforce a maximum length as a hard business rule.
 3. **Use `CHAR(n)` only** for truly fixed-width values: country codes, currency codes, fixed-format reference numbers.
 4. **Use CHECK constraints** for fine-grained validation: `CHECK (length(email) BETWEEN 5 AND 255 AND email LIKE '%@%')`.
 5. **Always use UTF-8** encoding for new databases.
 6. **Set collation at the database level** to match your primary locale; override at column level only when necessary.
 7. **Index for pattern matching** using `pg_trgm` extension for `LIKE '%pattern%'` queries.
 8. **Avoid `SELECT *`** on tables with large TEXT columns — fetch only required columns.
-

Performance Considerations

Indexes on text columns

```
-- Standard B-tree: fast for equality and prefix matching (LIKE 'prefix%')
CREATE INDEX idx_users_email ON users (email);

-- Trigram index: enables fast LIKE '%pattern%' and similarity search
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_users_name_trgm ON users USING GIN (full_name gin_trgm_ops);

-- Full-text search index
CREATE INDEX idx_articles_search ON articles USING GIN (search_vec);
```



```
-- Prefix search optimization
CREATE INDEX idx_users_username_prefix ON users (username varchar_pattern_ops);
```

Avoid functions on indexed columns

```
-- BAD: index on email cannot be used
SELECT * FROM users WHERE LOWER(email) = 'alice@example.com';

-- GOOD option 1: store email pre-lowercased
-- GOOD option 2: functional index
CREATE INDEX idx_users_email_lower ON users (LOWER(email));
SELECT * FROM users WHERE LOWER(email) = 'alice@example.com'; -- uses index

-- GOOD option 3: use citext extension
```

TOAST tuning

```
-- Change TOAST threshold (default: 2048 bytes)
-- Lower threshold: compress more aggressively (saves space, uses CPU)
ALTER TABLE articles ALTER COLUMN body SET STORAGE EXTENDED; -- compress+toast
(default)
ALTER TABLE articles ALTER COLUMN body SET STORAGE PLAIN; -- never
compress/toast
ALTER TABLE articles ALTER COLUMN body SET STORAGE EXTERNAL; -- toast but don't
compress
ALTER TABLE articles ALTER COLUMN body SET STORAGE MAIN; -- compress but
prefer inline
```

Interview Questions & Answers

Q1: What is the actual storage difference between `varchar(255)` and `text` in PostgreSQL?

A: None. Both use exactly the same variable-length storage mechanism. A `varchar(255)` column only stores as many bytes as the actual string (plus 1-4 bytes of length overhead), not 255 bytes. The only difference is that `varchar(255)` enforces a maximum length check at write time.

Q2: Why does `'AB'::char(5) = 'AB'::text` return TRUE?

A: PostgreSQL strips trailing spaces from `char(n)` values when comparing with other character types. The `char(n)` type semantics define trailing spaces as insignificant. This can cause subtle bugs when you expect spaces to be meaningful.

Q3: What is TOAST and when does it activate?

A: TOAST (The Oversized-Attribute Storage Technique) is PostgreSQL's mechanism for storing large values that don't fit in a standard 8 KB page. It activates when a value exceeds roughly 2 KB. TOAST compresses the value and/or moves it to a separate TOAST table, storing only an 18-byte pointer in the main row.

Q4: How would you implement case-insensitive email uniqueness in PostgreSQL?

A: Three options: (1) Store emails always lowercased via a CHECK constraint or trigger; (2) Create a unique index on `LOWER(email)`; (3) Use the `citext` extension which provides a case-insensitive text type and make the column `CITEXT UNIQUE`.

Q5: What collation should you use for a multi-language application?

A: Use `und-x-icu` (Unicode default locale via ICU) or `en-x-icu` for English-first applications. ICU collations are more portable and feature-rich than the OS-dependent libc collations. For specific locales (German, French, etc.), use the corresponding ICU collation at the column level.

Q6: What is the `pg_trgm` extension and when is it useful?

A: `pg_trgm` provides trigram-based similarity and fast substring matching. It splits strings into sequences of 3 characters (trigrams) and builds a GIN or GIST index. This enables fast `LIKE '%pattern%'` queries and fuzzy text search — things that normal B-tree indexes cannot support.

Q7: Why is `VARCHAR(255)` so common even in PostgreSQL?

A: It's a habit carried over from MySQL, where `VARCHAR(255)` used to have special storage optimizations. In PostgreSQL it is meaningless as a performance hint. Use `TEXT` with a CHECK constraint for cleaner semantics.

Q8: How do you search for strings that are similar (fuzzy match) in PostgreSQL?

A: Use `pg_trgm`'s `%` similarity operator or `<->` distance operator. Example: `SELECT * FROM products WHERE product_name % 'coffe'` finds 'coffee' even with a typo. The `similarity()` function returns a 0–1 score.

Q9: What happens to string indexes when you run `ALTER TABLE ... ALTER COLUMN ... TYPE`?

A: PostgreSQL rewrites the table and rebuilds all indexes on that column. This is a long-running operation that holds an `ACCESS EXCLUSIVE` lock. For large tables, use `ALTER COLUMN SET DATA TYPE ... USING` with a subsequent `REINDEX CONCURRENTLY`.

Q10: How do generated columns interact with text types?

A: Generated columns (PostgreSQL 12+) can store computed text values. They use `STORED` (not `VIRTUAL`). Useful for: storing a normalized/lowercased version of a column for indexing, storing a slug derived from a title, or materializing a full-text search vector.

Exercises with Solutions

Exercise 1

Design a `products` table where: the SKU is always 8 uppercase alphanumeric characters, the name is 3–200 characters, the description can be any length, and slugs are automatically generated.

Solution:

```
CREATE OR REPLACE FUNCTION make_slug(input TEXT)
RETURNS TEXT LANGUAGE SQL IMMUTABLE STRICT AS $$
    SELECT lower(regexp_replace(trim(input), '^[a-zA-Z0-9]+' , '- ', 'g'));
$$;

CREATE TABLE products (
```

```

product_id BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
sku         CHAR(8)        NOT NULL UNIQUE
           CHECK (sku ~ '^[A-Z0-9]{8}$'),
name        VARCHAR(200)   NOT NULL
           CHECK (length(trim(name)) >= 3),
slug        TEXT           NOT NULL UNIQUE GENERATED ALWAYS AS
(make_slug(name)) STORED,
description TEXT,
created_at  TIMESTAMPTZ    NOT NULL DEFAULT now()
);

```

Exercise 2

Write a query to find all users whose names contain accented characters (non-ASCII) and display the byte length vs character length.

Solution:

```

SELECT
    full_name,
    char_length(full_name) AS char_count,
    octet_length(full_name) AS byte_count,
    octet_length(full_name) - char_length(full_name) AS extra_bytes_for_accents
FROM users
WHERE octet_length(full_name) > char_length(full_name)
ORDER BY extra_bytes_for_accents DESC;

```

Exercise 3

Implement a search function that finds products by partial name match, using trigrams for fuzzy matching.

Solution:

```

CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_products_name_trgm ON products
    USING GIN (name gin_trgm_ops);

-- Fuzzy search function
CREATE OR REPLACE FUNCTION search_products(search_term TEXT, similarity_threshold
REAL DEFAULT 0.3)
RETURNS TABLE (product_id BIGINT, name TEXT, similarity REAL)
LANGUAGE SQL STABLE AS $$
    SELECT
        product_id,
        name,
        similarity(name, search_term) AS sim
    FROM products
    WHERE similarity(name, search_term) >= similarity_threshold
    ORDER BY sim DESC
    LIMIT 20;
$$;

```

```
SELECT * FROM search_products('coffe mug'); -- finds 'coffee mug' etc.
```

Cross-References

- [01_numeric_types.md](#) — numeric types for comparison
- [07_special_types.md](#) — tsvector and tsquery for full-text search
- [08_constraints.md](#) — CHECK constraints on character columns
- [../07_Indexes/](#) — B-tree, GIN, and trigram indexes for text
- [../03_Intermediate_SQL/](#) — string manipulation in queries